

3D Virtual Cricket Stadium With WalkThrough and Meta-Playing Mode

Ch. Rambhadra Kumar
Dept. of CSE, IIIT Vadodara
Vadodara, Gujarat, India
202311024@iiitvadodara.ac.in

Annepu Adarsh
Dept. of CSE, IIIT Vadodara
Vadodara, Gujarat, India
202311012@iiitvadodara.ac.in

Dheeraj MG
Dept. of CSE, IIIT Vadodara
Vadodara, Gujarat, India
202311027@iiitvadodara.ac.in

Abstract—This paper presents *Cricket Stadium* - a fully interactive 3D virtual environment which was built using *UNITY 3D* and was modeled using *BLENDER 3.6*. It consists of 2 types of modes : (i) It is a first - person Stadium walkthrough that uses Unity's *Character Controller*, (ii) This game is fully controlled using physics featuring an automated bowling machine that bowls balls to the bat, four phase *Quaternion.Lerp* bat swing, it also consists of Rigid body collision-driven hit mechanics, score is calculated based on distance (0/1/2/4/6 runs), boundary trigger detection, a full scene management for TextMeshPro UI, wicket Events. For physics we use Unity - 22 compatible *Rigid body.velocity* API. While development there are 10 technical challenges. For mid range hardware performance testing yields a stable ≥ 62 FPS. This project presents a practical integration of real time physics, 3D modelling HCI design at the understanding level and C# scripting.

Index Terms—Unity 3D, Cricket Simulation, Blender, Quaternion Animation, Interactive Simulation, First-Person Perspective, Rigid body Physics, Character Controller, C#, TextMeshPro

I. INTRODUCTION

Cricket has become a very popular game from the past few decades, and has an estimated fanbase worldwide 2.5 billion or more. There are some commercial titles for example Cricket 24, but these rely on proprietary engines and AI systems, these are also inaccessible and unavailable for undergraduate developers.

Cricket Stadium is a virtual VR prototype simulation built entirely on UNITY 3D with Blender. It is a model Stadium that tell about beginner to intermediate scope. Key **Contributions** are :

- Blender-authored stadium (<30K polygons) optimized thoroughly for real time rendering.
- Four-phase bat swing state machine via *Quaternion.Lerp* — Controllers like Animator controllers are not required.
- Distance-threshold scoring is calculated using boundary trigger colliders and physics.
- A catalog in 10 Unity development challenges and solutions.

II. RELATED WORK

Unity 3D is used in around 50% of mobile games worldwide [2], and the integration of physics enables rigid body simulation critical for ball-sport mechanics. Normally, most work was done on football and table tennis, but cricket simulation is underrepresented in open academic literature. Some

TABLE I
COMPARISON OF CRICKET GAMES VS. THIS PROJECT

Feature	Cricket 24	Real Cricket	ACA Stadium
Platform	Console/PC	Mobile	PC (Unity)
AI Bowler	Yes	Yes	No (Machine)
Custom Stadium	No	Low-poly	Blender FBX
First-Person	No	Partial	Full
Open/Educational	No	No	Yes

TABLE II
TECHNOLOGY STACK

Layer	Technology	Version
Game Engine	Unity 3D	2022.3 LTS
Scripting	C# / .NET 4.x	9.0
3D Modeling	Blender	3.6
UI Framework	TextMeshPro	3.0.6
Physics	Unity PhysX	Built-in
IDE	Visual Studio	2022

commercial cricket games (*Cricket 24*, *Don Bradman Cricket*) are based on motion-captured animations, and proprietary AI bowlers are not accessible and not available to academic developers.

Blender has become the standard open-source 3D authoring tool used for Unity pipelines [4]. It also requires key considerations such as polygon decimation, UV atlasing, and FBX export at a 1 unit = 1 metre scale. Feature comparisons are provided in Table I.

III. SYSTEM ARCHITECTURE

A. Technology Stack

B. Component Architecture and Data Flow

This system is constructed based on Unity's component-based architecture. *GameManager* is the central coordinator: this stores the active ball reference, and updates the TMP UI, then it mediates between *BallSpawner*, *BatController*, *BallController*, and *BoundaryCtrl* via *FindFirstObjectByType<T>()* and the direct method calls used here (Fig. 1). The game loop starts: *Start Game*

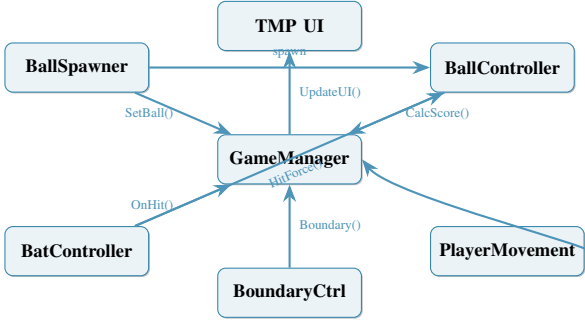


Fig. 1. Script dependency diagram.

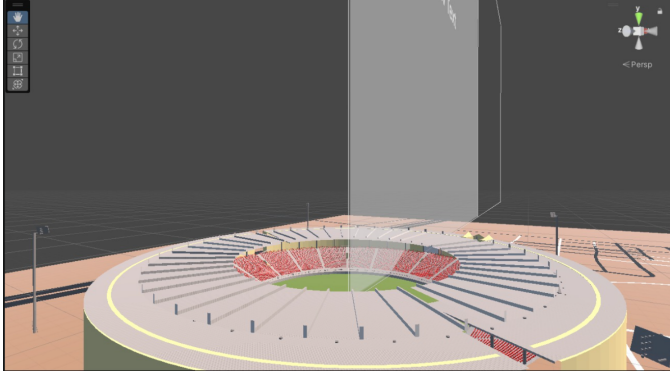


Fig. 2. Blender 3.6 viewport — complete ACA Cricket Stadium model.

TABLE III
STADIUM COMPONENTS — POLYGON BUDGET

Component	Polys	Material
Outer Stands	12,000	Concrete, glass
Seating Rows	8,500	Red plastic
Cricket Pitch	200	Turf + clay
Outfield Grass	400	Green shader
Floodlight Towers	2,400	Metallic steel
Boundary Rope	360	White PVC
Scoreboard	800	LED texture
Pavilion	3,200	Brick + glass
Roof Canopy	1,800	Corrugated steel
Total	29,660	—

→ BallSpawner Throws a ball every 5 seconds towards the player → player then presses **F** to swing the bat → OnCollisionEnter detects the "Ball" during the hit window → then impulse is applied → the landing distance is scored → UI is updated at last.

IV. 3D STADIUM MODELING

The ACA cricket stadium was done in blender from scratch in blnder 3.6 following a stardard oval layout (Fig. 2). we have done this blender all stadium before unity import: Decimate modifier on stands targeting <50K triangles; ambient-occlusion baked uv atlas; Fbx export at scal 1.0; imageplane nodes disabled. unity import settings: scale factor is 1.0,import normals=Import and this is how we have imported(Table III).

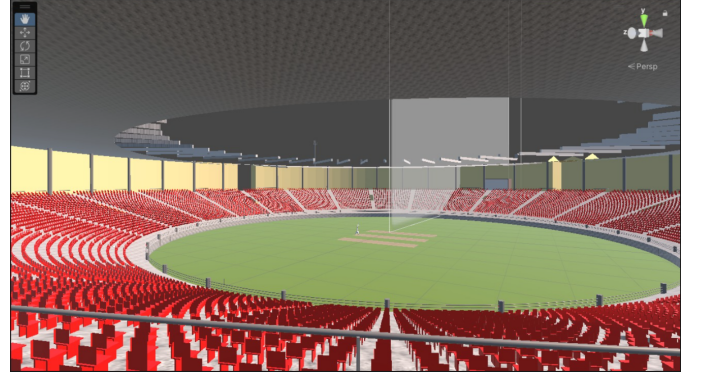


Fig. 3. First-person walkthrough view inside the stadium.

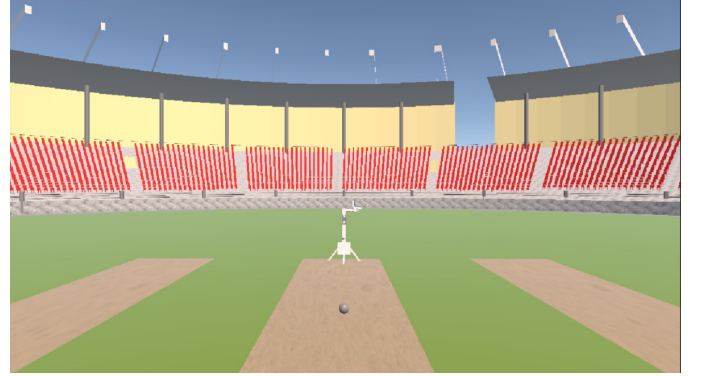


Fig. 4. first person view with incoming ball and wickets are in back side the batsman

V. IMPLEMENTATION

A. First-person walkthrough and player movement

In this walkthrough mode (Fig. 3), Unity's CharacterController (height 1.8 m and radius 0.4 m, speed 5.0 m/s, mouse sensitivity 2.0, pitch clamped $\pm 80^\circ$). Near clip plane set to 0.01 prevents bat clipping. WASD + mouse-look provides full spectator exploration of stands, pitch, and pavilion.

B. batting and bowling machine

The bowling machine (Z=18m) auto-spawns balls every 5s at 14.0 m/s if we want speed more we can increase it on code and with 1.2 units upward arc towards the player (Table IV). balls are auto destroyed after 6s if inhit. Fig. 4 shows the first-person batting perspective and To swing the bat the we press left-click to swing.

C. Bat swing

A four phase state machine (Fig. 5) uses *Quaternion.Lerp* — no animation controller required. OnCollisionEnter register and hit only during the *DownSwing* and *FollowThrough* phases; impulse of 28 N.s is applied in the cameras forward direction with positive y bias for areal trajectory (Table V).

TABLE IV
BALLSPAWNER CONFIGURATION

Parameter	Value	Description
Spawn interval	5.0 s	time between deliveries
Ball speed	14.0 m/s	Launch speed
Arc height	1.2	upside Y bias
machine z position.	18.0 m	distance from batsman
auto destroy	6.0 s	ball life span

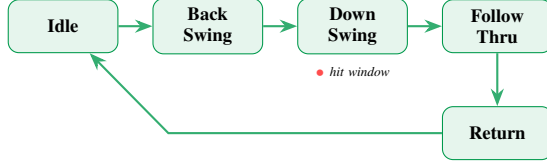


Fig. 5. Four-phase bat swing state machine.

TABLE V
BAT SWING PHASE PARAMETERS

Phase	Time	Euler Rot.	Purpose
Idle	—	(0, 0, 0)	Rest pos.
BackSwing	0.12 s	(-75, 15, 10)	Lift bat
DownSwing	0.09 s	(5, -10, -5)	Hit zone
FollowThru	0.18 s	(55, -20, -10)	Follow-thru
Return	0.28 s	→ Idle	Reset
Total	0.67 s	—	Full cycle

D. Scoring System and UI

the distance from where the projectile came from to where it lands via the Euclidean distance on the XZ-plane (Eq. 1), mapped to runs via Eq. 2. collides with the trigger cylinder of the projectile on the XZ-plane (*tag*="Boundary") while also awarding 4 runs when the projectile's trigger cylinder is crossed. *canvas* panels — *startpanel*, *gamepanel* (TMP score + instruction text), and *pausepanel* (toggled via *Escape*) — manage all UI state transitions.

$$d = \|\mathbf{p}_{\text{land}}^{xz} - \mathbf{p}_{\text{spawn}}^{xz}\|_2 \quad (1)$$

$$R(d) = \begin{cases} 0 & d < 8 \text{ m} \\ 1 & 8 \leq d < 18 \text{ m} \\ 2 & 18 \leq d < 35 \text{ m} \\ 4 & 35 \leq d < 52.5 \text{ m} \\ 6 & d \geq 52.5 \text{ m} \end{cases} \quad (2)$$

VI. RESULTS AND PERFORMANCE ANALYSIS

A. Gameplay Outcome Distribution

A Test session is also conducted which included 200 deliveries overall and produced an outcome distribution shown in Fig. 6. with about 32% dot balls, while sixes occurred at 6% the appropriate difficulty level.

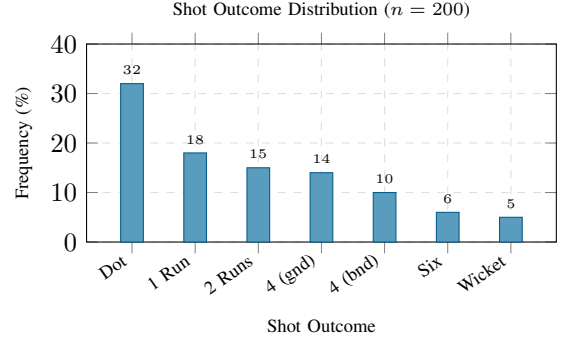


Fig. 6. Shot outcome distribution ($n = 200$ deliveries).

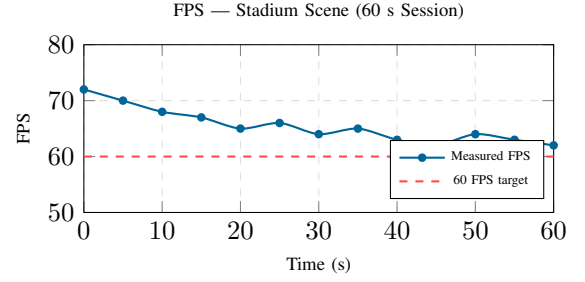


Fig. 7. Frame rate during active gameplay (mid-range hardware).

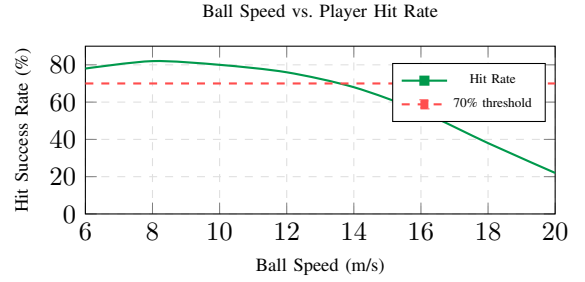


Fig. 8. Delivery speed vs. player hit success rate ($n = 50/\text{speed}$).

B. Frame Rate and Ball Speed Analysis

Here conducted operation for frame rates and Ball speed analysis. Where FPS remained ≥ 62 throughout a 60-second session on an Intel Core i5-10400 / NVIDIA GTX 1650 system (Fig. 7). It is a sensitivity study across bowling speeds overall (Fig. 8) and identified that 10–14 m/s as the optimal window for $\geq 70\%$. The default hit rate in the current 14m/s configuration is intended to find the parameters and are working well and very challenging to play. The batting statistics are presented together to both results in consistency and accuracy during 2 separate (60-ball) batting sessions Tables VII.

VII. DEVELOPMENT CHALLENGES AND SOLUTIONS

Table VI catalogs the 10 primary technical challenges encountered during development along with their root causes and resolutions.

TABLE VI
TECHNICAL CHALLENGES AND CORRESPONDING SOLUTIONS

#	Challenge	Root Cause	Solution
1	<code>rb.linearVelocity</code> compile error	<code>linearVelocity</code> added in Unity 6; not in Unity 2022 LTS	Replaced with <code>rb.velocity</code> throughout
2	Ball disappears immediately after the spawn	Destroy timeout too short; missing “Ground” tag on plane	Extended destroy time to 20 s; added tags; confirmed Sphere Collider
3	Score UI not updating	<code>OnBoundaryHit()</code> / <code>OnWicketHit()</code> defined outside class body	Rewrote complete <code>GameManager.cs</code> with all methods inside class scope
4	Bat not visible in the Game view	Near clip plane 0.3 culled bat mesh at close range	Set Main Camera near clip to 0.01
5	Ball flies away from player	<code>AddForce(camera.forward)</code> from player origin pushes ball away	BallSpawner at Z=18 m applies velocity toward player via direction vector
6	FBX bat offset (pCube1 X: -9.5 m)	Maya FBX export included off-centre pivot	Reset pCube1 Transform to (0,0,0); disabled imagePlane nodes
7	Duplicate bowling machine in scene	FBX model and empty BallSpawner GameObject created independently	Parented FBX model as child of the BallSpawner GameObject
8	BallSpawner namespace not found	<code>GameManager.cs</code> referenced type before script file created	Created <code>BallSpawner.cs</code> first; provided fallback <code>GameManager</code>
9	Crowd capsules spawning on pitch	<code>CrowdGenerator.startRadius</code> (45 m) too small for the stadium scale	Disabled generator; Blender model includes built-in seat geometry
10	Game screen frozen / input unresponsive	<code>CursorLockMode.Locked</code> trapped cursor without a Game view focus	Changed to <code>CursorLockMode.Confined</code> ; instructed click inside Game view

TABLE VII
BATTING STATISTICS — 60-BALL TEST SESSIONS

Metric	Session 1	Session 2
Balls Faced	60	60
Dot Balls	21	18
Singles	12	14
Twos	9	11
Fours (ground)	10	9
Fours (boundary)	5	6
Sixes	3	2
Total Runs	87	91
Strike Rate	145.0	151.7

VIII. FUTURE WORK

Short-term priorities will include an AI bowler with parameterized delivery variation (speed, seam, yorker), a full innings system with over count and a run rate. Medium-term will targets the cover crowd audio triggers, a day/night cycle with animated floodlights, and a weather system affecting pitch behavior. Long-term goals will include the multiplayer via Photon PUN2 and the Unity XR Toolkit VR integration. At present architecture supports the these extensions: `BallSpawner` accepts per-delivery speed variation at the runtime and `GameManager.CalculateScore()` can be extended with the LBW and the run-out events.

IX. CONCLUSION

Cricket Stadium successfully integrates a <30K polygon Blender stadium, a four-phase quaternion bat swing, a configurable bowling machine, a distance-threshold scoring the

engine with a boundary and with a wicket detection, and a three-panel `TextMeshPro` UI — all within Unity 2022 LTS. Performance testing confirmed the stable ≥ 62 FPS on mid-range hardware, and an optimal delivery speed of 12–14 m/s yielding $\geq 70\%$ hit rate. The systematic catalog of 10 development challenges provides the reproducible guidance for the Unity 2022 game development education. Our project establishes a solid and String foundation for the AI bowling, innings management, multiplayer, and VR extensions.

ACKNOWLEDGMENT

We really thank our project guide’s for supervision throughout development. 3D models (bowling machine, cricket bat) sourced from Sketchfab under Creative Commons license. Stadium geometry modeled originally in the Blender 3.6.

REFERENCES

- [1] International Cricket Council, “ICC Global Cricket Development Report,” *ICC Publications*, 2023. [Online]. Available: <https://www.icc-cricket.com>
- [2] Unity Technologies, “Unity Engine — Official Documentation,” 2023. [Online]. Available: <https://docs.unity3d.com>
- [3] J. Hocking, *Unity in Action: Multiplatform Game Development in C#*, 2nd ed. Shelter Island, NY: Manning, 2018.
- [4] Blender Foundation, “Blender 3.6 Reference Manual,” 2023. [Online]. Available: <https://docs.blender.org/manual/en/3.6/>
- [5] Unity Technologies, “Physics Best Practices,” Unity Manual, 2023. [Online]. Available: <https://docs.unity3d.com/Manual/PhysicsBestPractices.html>
- [6] Unity Technologies, “TextMeshPro User Guide,” Unity Packages, ver. 3.0.6, 2023. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/>
- [7] Sketchfab Inc., “3D Model Repository — Cricket Assets,” 2024. [Online]. Available: <https://sketchfab.com>
- [8] Unity Technologies, “CharacterController — Unity Scripting API,” 2023. [Online]. Available: <https://docs.unity3d.com/ScriptReference/CharacterController.html>